



MOBILE APPLICATION-BASED TAXI METER

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING

FACULTY OF ENGINEERING

UNIVERSITY OF RUHUNA

AN UNDERGRADUATE PROJECT PROPOSAL
SUBMITTED IN PARTIAL FULFILMENT OF THE DEGREE OF THE
BACHELOR OF THE SCIENCE OF ENGINEERING

ON

07TH NOVEMBER 2025

EE5206: SOFTWARE PROJECT

GROUP MEMBERS

HAMID K. K. J. : EG/2022/

MUNASINGHE M. A. B. M. : EG/2022/5198

PERERA M. T. J. P. : EG/2022/5239

RODRIGO W. T. N. : EG/2022/5297

.....

MR. SUBODHA GUNAWARDENA

ABSTRACT

This project proposes the development of a Mobile Application-Based Taxi Meter System designed to simplify and digitalize taxi fare calculation while promoting transparency between taxi drivers and passengers. Each registered taxi driver will receive a unique QR code issued by the government. Customers can scan this QR code to view the driver's per-kilometer rate, compare rates across areas, and negotiate fares before confirming a ride.

The system will allow taxi drivers to set or update their per-kilometer rates and automatically generate trip receipts after each journey. Receipts will be securely stored in a database for future reference and income tracking. This database is an alternative option where the admin can activate or deactivate, as this option can increase the data transactions, hence can be inconvenient to the users. Additionally, the app will compute average per-kilometer prices in specific regions, visually indicating whether the current rate is above or below average.

The proposed system will prioritize minimal data interaction to ensure fast responses and offline functionality, allowing continued use even during central server outages. This project aims to enhance transparency, efficiency, and security in local taxi operations through an easy-to-use and data-light mobile platform.

ACKNOWLEDGEMENT

We extend our gratitude to our client, Dr. Subodha Gunawardena, for providing valuable insights and requirements that guided the direction of this project.

Contents

1	Introduction.....	1
1.1	Background	1
1.2	problem statement	1
1.3	objectives and scope	1
1.4	Methodology	2
1.5	Time plan and resources required.....	2
1.5.1	Time plan.....	2
1.5.2	Resources required.....	3
2	Requirements	4
2.1	Functional requirements	4
2.2	Technical requirements	4

1 INTRODUCTION

1.1 BACKGROUND

Traditional taxi services often rely on mechanical or digital taxi meters that are costly to install and maintain. Meanwhile, modern taxi-meter apps charge high commissions, reducing drivers' profit margins. Many independent drivers lack access to affordable, transparent systems to display their fare rates to customers. A mobile application that acts as a smart taximeter can overcome these limitations by providing a convenient and flexible solution for both drivers and passengers.

1.2 PROBLEM STATEMENT

The absence of a unified, commission-free, and transparent system prevents drivers from independently setting their rates while maintaining customer trust. Customers, on the other hand, face difficulty verifying the authenticity of drivers and their fare rates. Furthermore, server-dependent applications often fail during connectivity issues, limiting reliability. Therefore, there is a need for a lightweight, offline-capable mobile app that provides QR-based verification, rate visibility, and automated fare calculation.

1.3 OBJECTIVES AND SCOPE

Objectives:

- Develop a mobile app that generates and assigns a unique QR code to each registered taxi driver.
- Enable customers to scan the QR code and view the driver's rate and details.
- Allow drivers to set or update their per-kilometer rate dynamically.
- Implement a receipt generation system with a secure storage mechanism.
- Display area-wise average taxi rates with color indicators (red for high, green for normal).
- Support offline operation with local fare calculation.

Scope:

- The system will be implemented as a mobile application with optional web dashboard for government regulators.
- A local database will store receipts and driver data, with optional synchronization to a central database.
- The app will provide minimal data interaction for quick response and efficient performance.

1.4 METHODOLOGY

1. Mobile Application Development: The app will be developed using Flutter or React Native for cross-platform compatibility. Core features include QR scanning, rate setting, trip tracking, and receipt generation.
2. Database Management: A local SQLite database will handle offline data storage. Optional synchronization with a cloud database (Firebase or MongoDB Atlas) will be included for central data access.
3. Backend and APIs: A lightweight Node.js/Express backend will manage user authentication, rate storage, and government verification.
4. Testing and Deployment: Functional and performance testing will ensure quick response and reliable offline use. The final product will be deployed on Android devices, with web access for regulators.

1.5 TIME PLAN AND RESOURCES REQUIRED

1.5.1 Time plan

Week	To be done
1 – 3	Frontend mobile app development
4 – 7	Backend and database development
8 – 10	QR generation and scanning integration
11 – 14	Receipt generation and data storage
15 – 18	Offline functionality and optimization
19 – 21	Testing and debugging
22 – 24	Documentation and presentation preparation

1.5.2 Resources required

- Development platforms: VS Code
- Technologies: Flutter, Node.js and Express, SQLite/Mongo DB
- Devices: Android smartphones for testing
- Internet connection for synchronization and deployment

2 REQUIREMENTS

2.1 FUNCTIONAL REQUIREMENTS

1. Driver Registration & Verification: Government registration and QR code generation; driver details stored securely.
2. QR Code Scanning: Customer scans driver QR code to view rate and profile.
3. Rate Management: Driver sets or updates per-kilometer rate; customers can view and negotiate.
4. Trip & Receipt Management: Automatic fare calculation and digital receipt generation.
5. Area-wise Rate Analysis: Average rate calculation and color indicators (red for high, green for normal).
6. Offline Functionality: Local fare computation and receipt generation.
7. Regulator Dashboard (optional): Manage driver accounts and monitor rates.

2.2 TECHNICAL REQUIREMENTS

- Frontend : Flutter
- Backend : Node.js + Express
- Database : SQLite (offline) + MongoDB (online)
- QR Code Generation : QR Flutter / QR.js
- Authentication : JWT or Firebase Auth
- Hosting : Microsoft Azure
- Tools : Android Studio, Postman, Figma for UI design